

# (5/17/2026) How a Windows Kernel Mitigation Ruined My Memory Leak - Ben Lye

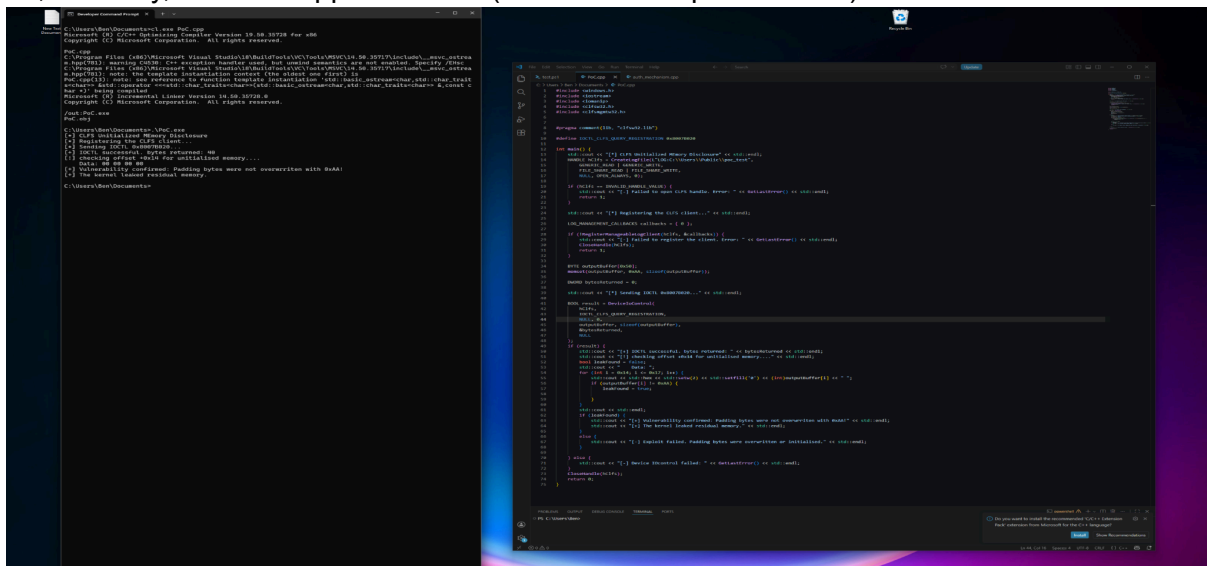
## Background

It's something I have ran into a couple times now where you see a code flaw after running the .sys driver through Ghidra and you see some memory regions haven't been initialised (as seen here from 0x13 to 0x18) and it turns out to be an absolute nothing-burger. Granted, lots of fun, but I thought this write up would add value to those who are interested in how umbrella Kernel mitigations, and in this case the Windows I/O Manager, neutralised struct alignment gaps in the CLFS.sys driver I was looking into.

So, firstly I want to disclose my actual motive for having a gander at this. Despite my title, I am a pentester by trade, not a vulnerability researcher, and this is something I have been largely in my own time for interests sake.

While I was auditing the dispatch routine of CLFS.sys (technically the secondary dispatch routine) I discovered a code defect that a missing structure member initialisation left a 4-byte alignment padding gap unhandled deeply nested within the CLFS.sys driver. Naturally, I got really excited, I was thinking of all the ways I would be able to try and leak this into the user space to perhaps try and get a KASL bypass or something along those lines as on paper, this looked like an absolute bread-and-butter information disclosure vulnerability capable of leaking kernel pool memory to the user-space.

So, naturally, I wrote a cpp POC on it (as seen in the photo below).



I found a really interesting feature of the Windows Kernel (which, granted, is documented) that I had absolutely no idea existed before as it hadn't always existed (was my understanding).

## Discovery

The defect itself actually sits within the `CClfsManagedLogClientUser::QueryRegistration` function. Once put through Ghidra (and after manually mapping the structs as this is not part of the pdb and without it the offsets are unclear), I was able to see how the driver handles returning registration data back to a calling user-mode application.

The function populates a data structure and copies it into an output buffer, however, the structure contains fields of varying data types (like pointers and 32bit ints). Because the C++ compiler enforces rigid guidelines around memory alignment for 64bit architectures, it automatically inserts invisible padding bytes between the structure members to ensure that 64-bit values align to 8-byte boundaries in memory.

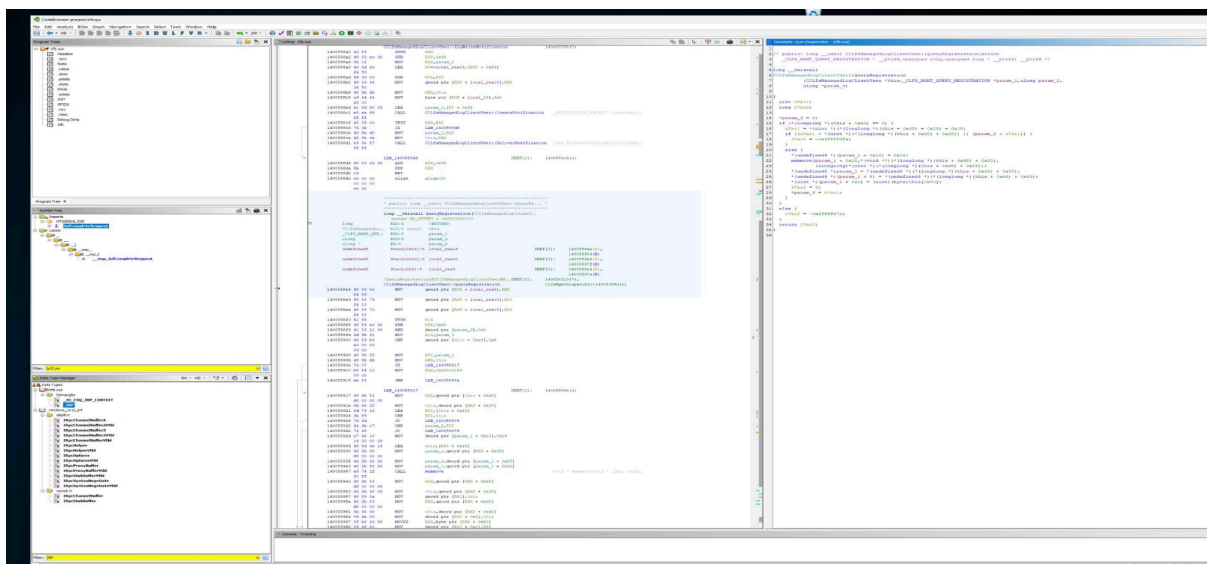
When the driver populates this memory, it explicitly assigns values to the designated structure members but completely skips assigning a value to the compiler-generated gap.

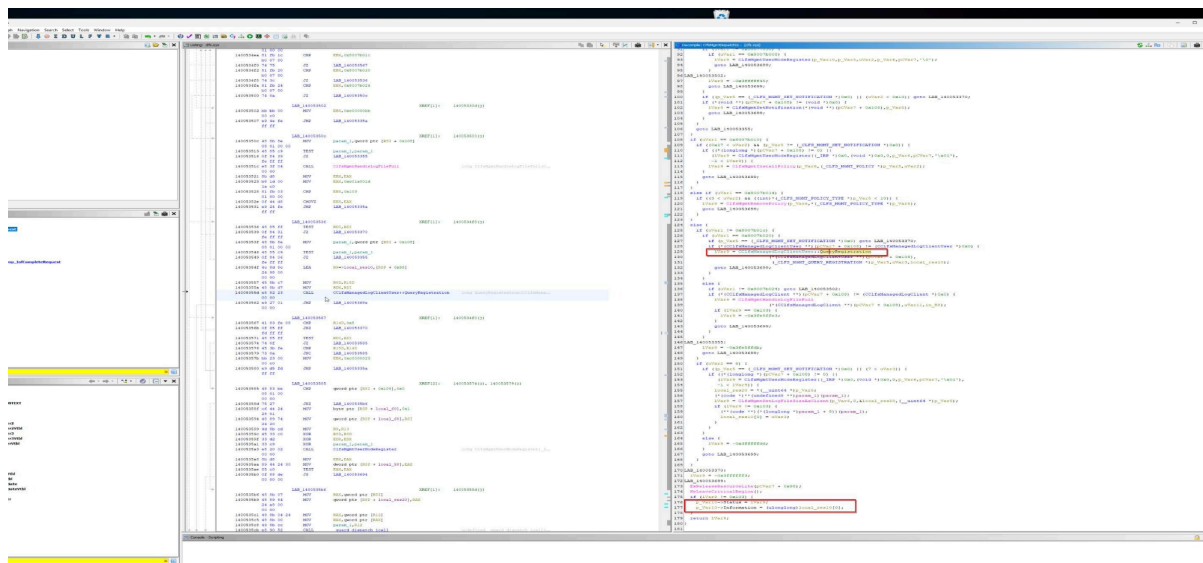
Have a look at this:

- Driver itself writes data to offset 0x00
- It writes to offset 0x08
- It writes to offset 0x0C
- It writes to offset 0x10
- It completely SKIPS writing to the 4-byte gap spanning offsets 0x14 through to 0x17

If this buffer were allocated directly from the active kernel heap and copied straight into the user space the 4 skipped bytes would contain raw kernel memory that was previously occupying that heap chunk which is a kernel information disclosure primitive.

Below is two images outlining the actual part of the .sys driver I'm talking about in Ghidra.





## The IO/Manager Safety Net

This is literally the only reason I'm disclosing this over this forum and not taking it to Microsoft. When executing the compiled binary, the IOCTL succeeds in passing through all driver validations and returning a successful execution frame. However, examining the target output bytes gives us uniform results across every platform that I actually tested this on.

```
C:\Users\Ben\Documents>cl.exe PoC.cpp
Microsoft (R) C/C++ Optimizing Compiler Version 19.50.35728 for x86
Copyright (C) Microsoft Corporation. All rights reserved.

PoC.cpp
C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Tools\MSVC\14.50.35717\include\__msvc_ostrea
m.hpp(781): warning C4538: C++ exception handler used, but unwind semantics are not enabled. Specify /EHsc
C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Tools\MSVC\14.50.35717\include\__msvc_ostrea
m.hpp(781): note: the template instantiation context (the oldest one first) is
PoC.cpp(13): note: see reference to function template instantiation 'std::basic_ostream<char,std::char_traits
<char>> &std::operator <<std::char_traits<char>>(std::basic_ostream<char,std::char_traits<char>> &,const c
har *)' being compiled
Microsoft (R) Incremental Linker Version 14.50.35728.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:PoC.exe
PoC.obj

C:\Users\Ben\Documents>.\PoC.exe
[*] CLFS Initialized MEmory Disclosure
[*] Registering the CLFS client...
[*] Sending IOCTL 0x8007B020...
[+] IOCTL successful. bytes returned: 40
[!] checking offset +0x14 for uninitialised memory....
Data: 00 00 00 00
[+] Vulnerability confirmed: Padding bytes were not overwritten with 0xAA!
[+] The kernel leaked residual memory.

C:\Users\Ben\Documents>
```

And I admit I did not know this before, but why the 0xAA values were wiped out and replaced with pure zeroes (not random kernel data) has to do with how the Windows I/O sub-system inside the main kernel binary (ntoskrnl.exe) works.

The IOCTL code defined by the driver (0x8007B020) is responsible for setting how the operating system must bridge user-space memory with kernel-space memory (which is obviously a very high risk boundary). Actually looking deeper into this constant shows us that the least significant two bits are 00 which in WDM hardcodes the request to use METHOD\_BUFFERED (can find this online if you are unsure of the API call).

Since it is buffered, clfs.sys doesn't control or manage the actual allocation of the buffer it is writing to and instead the upstream Windows I/O manager intercepts the request, sorts out the data allocation, and passes a standardized pointer down to the target driver via AssociatedIrp.SystemBuffer.

When clfs.sys handles a METHOD\_BUFFERED request (or any other driver) a single kernel

allocation serves as both the input and output container, and to figure out how much space to provision the I/O manager actually then evaluates the transaction using this mathematical boundary ( $\text{Allocation Size} = \max(\text{InputBufferLength}, \text{OutputBufferLength})$ ).

Since our PoC requests an output of size 0x50 bytes but provides an input of 0 the I/O Manager allocates a 0x50 -byte chunk from the kernel pool. Brilliantly, to eliminate the exact thing I found here with an umbrella sweep, the Windows I/O manager zero-initialises any trailing region of the newly allocated buffer that isn't actively occupied by incoming data, a really smart feature IMO, and something I was unaware of honestly..